
Unsupervised Neural Langevin Sampler for Mixed Integer Linear Programming

Yixin Huang¹ Shengyu Feng¹ Yiming Yang¹

Abstract

Existing neural combinatorial optimization (CO) solvers often rely heavily on expensive labeled data and additional post-processing to produce feasible solutions. Research into mixed integer linear programs (MILPs) is particularly limited due to the lack of effective heuristics for feasibility and the challenge of modeling mixed-type variables for neural solvers. To address these issues, we propose a novel unsupervised Langevin sampler for solving MILPs. Our framework learns only integer variables, while continuous variables are solved using an exact linear programming solver, thus isolating the combinatorial hardness of the problem and avoiding unnecessary modeling complexity. The sampler is based on Langevin dynamics and incorporates both objective optimization and constraint satisfaction into a unified energy function, enabling the model to jointly learn feasibility and optimality. Experiments demonstrate that our method achieves 100% feasibility without expensive post-processing and matches or outperforms supervised solvers on benchmark datasets, highlighting its effectiveness and scalability. Our code can be found at <https://github.com/CindyH1103/UNLS4MILP>.

1. Introduction

A Combinatorial Optimization (CO) problem (Wolsey & Nemhauser, 1999) is an optimization problem whose goal is to find optimal solutions from a discrete space. CO problems arise in a wide range of real-world applications, including facility location (Charikar & Guha, 1999), vehicle routing (Delarue et al., 2020), and job scheduling (Ku & Beck, 2016). Despite their broad applicability, CO problems are typically NP-hard, making them computationally

challenging to solve at scale. Traditional methods such as branch-and-bound (Wolsey, 2020), cutting-plane methods (Ceria et al., 1998), and large neighborhood search algorithms (Ahuja et al., 2002) are typically computationally expensive, especially for large-scale problems.

With advances in machine learning, recent studies have increasingly turned to neural networks for help. In addition to vanilla supervised learning (Gasse et al., 2019), applications of more advanced methods such as diffusion (Sun & Yang, 2023b) and Langevin dynamics (Feng & Yang, 2025) all show promising performance. However, two major limitations hinder the broader and more generalizable applications of machine-learning techniques to CO problems:

- **Dependence on near-optimal supervision.** Most neural solvers rely heavily on supervised training signals generated by powerful classical solvers. This dependence restricts their applicability to problems where high-quality solutions are not readily available.
- **Dependence on human-designed heuristics for feasibility.** Existing neural solvers typically rely on hand-crafted heuristics, such as greedy decoding, to convert model output into feasible solutions. This reliance limits their applicability to problems that lack effective or well-established heuristic procedures, e.g., general mixed integer linear programs.

Moreover, most existing neural solvers are designed for CO problems with exclusively integer variables, as the presence of continuous variables introduces additional feasibility constraints and structural complexity. However, many real-world optimization problems naturally involve both discrete and continuous variables and are most naturally formulated as mixed integer linear programs (MILPs). Despite the complexity of MILP, one key characteristic of MILP is that the NP-hard nature of MILP arises solely from the integer space. Once integer assignments are fixed, the continuous ones can be efficiently determined by solving a linear program (LP) using exact LP solvers. Consequently, explicitly modeling continuous variables within neural networks often provides limited additional benefit, while substantially increasing model complexity and training instability. One question naturally arises: can we exploit this structure to solve general MILPs in an unsupervised manner without

¹Carnegie Mellon University. Correspondence to: Yixin Huang <yixinhu2@cs.cmu.edu>.

relying on expensive heuristic repair or explicit modeling of continuous variables?

In this paper, we propose a novel unsupervised framework that formulates MILP as a decomposed sampling problem. The integer variables are modeled and updated by the Langevin neural network, while the corresponding continuous variables are obtained by solving the resulting LP. To facilitate unsupervised learning, we adopt a learned Langevin dynamics-based model (Feng et al., 2026), which provides a principled mechanism to explore the discrete solution space under the guidance of an energy function that encodes both objective value and constraint satisfaction. The envelope theorem (Milgrom & Segal, 2002) further provides gradients of the energy function, allowing gradient-based optimization of the model parameters.

We evaluated this method on three classic MILPs. Despite not having direct supervision, our method performs similarly to or even better than other supervised methods. In contrast to diffusion-based solvers that require a large number of sampling steps, our method is significantly more time-efficient, producing high-quality approximate solutions in only a few iterations. Furthermore, our method consistently yields feasible solutions without relying on costly post-processing or repair heuristics.

2. Related Works

Traditional solvers for MILP. Traditional MILP solvers include branch-and-bound (Wolsey, 2020), cutting-plane (Ceria et al., 1998) methods and Benders decomposition (Rahmaniani et al., 2017). The branch-and-bound method systematically divides the solution space into subproblems and eliminates infeasible solutions based on bounds. Cutting-plane methods iteratively refine the feasible region by adding linear inequalities, thus reducing the search space. Benders decomposition divides variables into two subsets so that a first-stage master problem is solved over the first set of variables, and the values for the second set of variables are determined in a second-stage subproblem for a given first-stage solution. Given feedback from the subproblem, the master problem is iteratively re-solved until convergence. All traditional solvers suffer from scalability issues.

Neural Solvers for CO Combinatorial optimization has attracted a great deal of attention from the machine learning (ML) community (Zhang et al., 2023). One line of research seeks to replace ML models with key components of traditional algorithms to improve computational efficiency. A significant portion of this work focuses on learning heuristic policies for tasks such as selecting variables to branch out on (Scavuzzo et al., 2024), choosing cutting planes (Puigdemont et al., 2024), and more (Labassi et al., 2022). Those methods are still highly dependent on traditional algorithms,

and hence their performance is constrained by the performance of those traditional solvers.

Another line of research leverages ML models to predict solutions and adopts traditional methods as post-processing techniques to retrieve feasible solutions. For example, FMIP (Li et al., 2026) proposes to use a joint continuous-integer flow to capture the solution distributions. Though both integer and continuous variables are predicted, feasibility isn’t guaranteed, and it relies heavily on post-processing techniques, e.g., neural diving (Nair et al., 2021). Considering the costs of labeling data, many works also adopt an unsupervised manner for training. DiffUCO (Sanokowski et al., 2024) uses an energy-based diffusion model to find an optimal solution. However, general MILP cannot be transformed into a Quadratic Unconstrained Binary Optimization problem, which is the key assumption in DiffUCO’s formulation. DiffILO (Geng et al., 2025) reformulates integer linear programs (ILP) into continuous differentiable and unconstrained optimization problems and solves the problem with straightforward gradient descent, making a fully end-to-end solver for general ILPs. If continuous variables were to be introduced into DiffILO, such an approach would treat integer and continuous variables homogeneously. In contrast, our framework leverages the structural hierarchy of MILPs by partitioning the variable space. This decomposition not only preserves the distinct mathematical roles of each variable type but also enables our model to yield higher-quality solutions by focusing search on the discrete backbone of the problem. In general, there are only limited works that focus on neural solvers for MILP. We believe that our work will bridge this gap.

3. Preliminary

3.1. Combinatorial Optimization Problem

Based on the formulation of Papadimitriou & Steiglitz (1982), a general combinatorial optimization (CO) problem can be seen as a type of constrained optimization problem,

$$\min_{\mathbf{x} \in \{0,1\}^N} a(\mathbf{x}) \quad \text{s.t.} \quad b(\mathbf{x}) \geq 0, \quad (1)$$

where $a(\mathbf{x})$ is the objective function to be minimized and $\max(-b(\mathbf{x}), 0)$ measures the extent of constraint violation. This constrained optimization problem can be further rewritten into the following penalty form:

$$\min_{\mathbf{x} \in \{0,1\}^N} H(\mathbf{x}) = a(\mathbf{x}) + \beta \max(-b(\mathbf{x}), 0), \quad (2)$$

where $\beta > 0$ is the penalty coefficient that should be sufficiently large to ensure that the optimum of this formulation aligns with the original formulation shown in Equation 1. $H(\mathbf{x})$ is generally referred to as the energy function, and

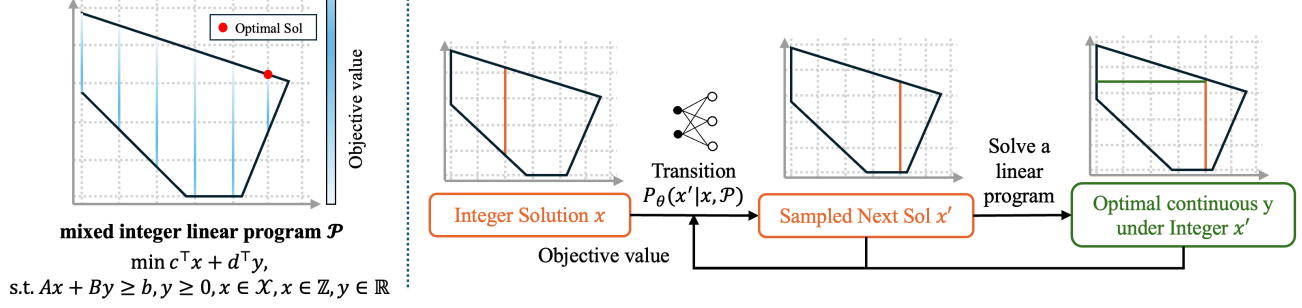


Figure 1. Visualization of the proposed unsupervised framework for solving MILPs. The left part illustrates the problem formulation and decomposition of MILP. The right part depicts the training pipeline of the proposed neural Langevin sampler.

the corresponding energy-based model (EBM) is defined as

$$p_\tau(\mathbf{x}) = \frac{\exp(-H(\mathbf{x})/\tau)}{Z}, \quad (3)$$

where $\tau > 0$ is the temperature that controls the smoothness of $p_\tau(\mathbf{x})$ and $Z = \sum_{\mathbf{x} \in \{0,1\}^N} \exp(-H(\mathbf{x})/\tau)$ is the normalization factor. When τ is small, the probability mass of p_τ tends to concentrate around low-energy samples, making solving Equation 1 be equivalent to sampling from $p_\tau(\mathbf{x})$. Markov Chain Monte Carlo (MCMC) (LeCun et al., 2006) is the most popular method for sampling from such EBMs.

3.2. Langevin Dynamics

Langevin dynamics (LD) (Welling & Teh, 2011) is a powerful MCMC algorithm for the continuous domain. It functions by performing a noisy gradient ascent to progressively move samples toward regions of higher log-likelihood. The update rule at each step is defined as

$$\mathbf{x}' = \mathbf{x} + \frac{\alpha}{2} \nabla \log p(\mathbf{x}) + \sqrt{\alpha} \zeta, \quad \zeta \sim \mathcal{N}(0, \mathbf{I}_{N \times N}), \quad (4)$$

where $\alpha > 0$ is the step size. Through iterative updates, the sample $\mathbf{x}$ would eventually converge to a stationary distribution that approximates $p(\mathbf{x})$.

Based on this, Zhang et al. (2022) extended the LD to discrete domains. We can reframe the update in Equation 4 as a transition distribution:

$$q(\mathbf{x}'|\mathbf{x}) = \frac{\exp(\frac{1}{2} s(\mathbf{x})^T (\mathbf{x}' - \mathbf{x}) - \frac{1}{2\alpha} \|\mathbf{x}' - \mathbf{x}\|_2^2)}{Z(\mathbf{x})}. \quad (5)$$

where $s(\mathbf{x}) = \nabla \log p(\mathbf{x})$. The above distribution allows coordinate wise factorization, i.e., $q(\mathbf{x}'|\mathbf{x}) = \prod_{i=1}^N q(\mathbf{x}'_i|\mathbf{x})$, where each $\mathbf{x}'_i$ follows a categorical distribution. The formulation could be further simplified for a binary state space:

$$q(\mathbf{x}'_i = 1 - \mathbf{x}_i|\mathbf{x}) = \sigma\left(\frac{1}{2} s(\mathbf{x})_i (1 - 2\mathbf{x}_i) - \frac{1}{2\alpha}\right), \quad (6)$$

where $\sigma(\cdot)$ stands for the sigmoid function.

3.3. Mixed Integer Linear Programs

Mixed integer linear programs (MILP) represent a broad and practically important extension of CO, in which discrete decisions are coupled with continuous variables through linear constraints and a linear objective:

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{X}, \mathbf{y} \geq 0, \mathbf{x} \in \mathbb{Z}^p, \mathbf{y} \in \mathbb{R}^{n-p}} \quad & \mathbf{c}^\top \mathbf{x} + \mathbf{d}^\top \mathbf{y}, \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} \geq \mathbf{b} \end{aligned} \quad (7)$$

with $\mathbf{c} \in \mathbb{R}^p, \mathbf{d} \in \mathbb{R}^{n-p}$ the objective coefficients, $\mathbf{A} \in \mathbb{R}^{m \times p}, \mathbf{B} \in \mathbb{R}^{m \times (n-p)}$ the constraint coefficient matrices, $\mathbf{b} \in \mathbb{R}^m$ the constraint right-hand side, $\mathcal{X}$ the feasible set for $\mathbf{x}$ and $p \leq n$, the number of integer variables.

3.4. Benders Decomposition

Benders decomposition is a classic algorithm for solving MILP (Rahmaniani et al., 2017). The method partitions the problem into a master problem over the integer variables and a subproblem over the remaining continuous variables. The master problem proposes candidate integer solutions, which are then evaluated in the subproblem to determine feasibility and values. Feedback from the subproblem is used to iteratively refine the master problem, guiding it toward globally valid and optimal solutions.

Notice that for Eq. 7, given any fixed $\bar{\mathbf{x}}$, the subproblem can be written as:

$$\min_{\mathbf{y} \geq 0} \mathbf{c}^\top \bar{\mathbf{x}} + \mathbf{d}^\top \mathbf{y}, \text{ s.t. } \mathbf{B}\mathbf{y} \geq \mathbf{b} - \mathbf{A}\bar{\mathbf{x}}, \mathbf{y} \in \mathbb{R}^{n-p} \quad (8)$$

Directly using the primal solution for feedback is insufficient because the resolved $\mathbf{y}$ is only valid for a given $\bar{\mathbf{x}}$ and does not provide information on other potential integer solutions. To generate globally valid constraints for the master problem, we consider the dual of the subproblem:

$$\max_{\mathbf{u} \geq 0} (\mathbf{b} - \mathbf{A}\bar{\mathbf{x}})^\top \mathbf{u} + \mathbf{c}^\top \bar{\mathbf{x}}, \text{ s.t. } \mathbf{B}^\top \mathbf{u} \leq \mathbf{d}, \mathbf{u} \in \mathbb{R}^m \quad (9)$$

Every feasible dual $\mathbf{u}$ helps to construct a globally valid constraint on the master problem. The master problem then

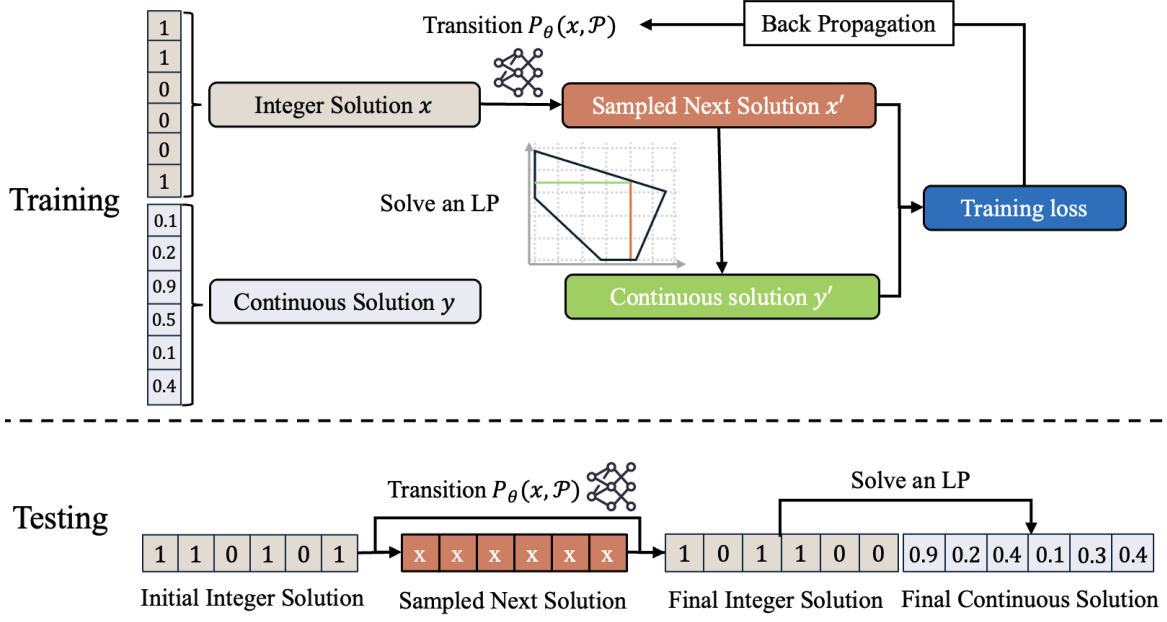


Figure 2. Illustration of the proposed training and testing paradigm

optimizes over integer variables $\mathbf{x}$ and an auxiliary variable $\mathbf{z}$ that approximates the objective function.

$$\min_{\mathbf{x}, \mathbf{z}} \mathbf{z}, \quad \text{s.t. } \{\text{cuts}\}, \mathbf{x} \in \mathcal{X} \quad (10)$$

Solving the master problem provides a lower bound on the MILP objective, while evaluating the subproblem at the proposed integer solution provides an upper bound. The algorithm iterates until the gap between the upper and lower bounds is sufficiently small or until infeasibility is detected.

If we assume that the problem in concern has a finite optimal solution, then the subproblem will not be infeasible. Hence, two types of cuts are introduced based on the subproblem solution. If the subproblem has a finite optimal solution $\bar{\mathbf{u}}$, the master problem is constrained to consider only integer solutions that improve upon the current objective: $\mathbf{z} \geq (\mathbf{b} - \mathbf{A}\bar{\mathbf{x}})^\top \bar{\mathbf{u}} + \mathbf{c}^\top \mathbf{x}$. This will help to strictly increase the value of $\mathbf{z}$ at the solution $\bar{\mathbf{x}}$.

If the subproblem is unbounded, the corresponding integer assignment is infeasible. An extreme ray $\bar{\mathbf{u}}$ that certifies the unboundedness is used to construct a constraint to exclude the current integer assignment: $(\mathbf{b} - \mathbf{A}\mathbf{x})^\top \bar{\mathbf{u}} \geq 0$.

4. Methodology

Previous works have addressed the application of Langevin Dynamics (LD) to both continuous variables (Welling & Teh, 2011) and discrete variables (Zhang et al., 2022). However, how to apply it to mixed-variable cases remains largely underexplored. Here, we propose a novel method to apply LD to MILPs, which is visualized in Fig. 2.

In general, the proposed solver operates as a stochastic search of integer variables guided by LP-based evaluation. Our goal is to find a solution that minimizes the energy function defined in Eq. 2. We adopt Langevin dynamics as a stochastic optimization method to approximately sample low-energy solutions.

A key design choice is to restrict the sampling process to integer variables (Section 4.1). Given an integer assignment, the continuous variables are solved via an LP subproblem. During inference, a learned neural network parameterizes this sampling process.

The training procedure focuses on minimizing the expected energy after a single transition step. To achieve this, we introduce three training objectives (Section 4.4). For gradient-based objectives, we derive subgradient signals for integer variables from the dual solutions of the corresponding LP. Sections 4.2 and 4.3 detail the treatment of feasible and infeasible LPs, respectively, allowing the calculation of meaningful update directions in both cases and ensuring a well-defined optimization process throughout the training.

4.1. Reformulating MILP via Benders Decomposition

Once the discrete variables in an MILP are fixed, the remaining optimization over continuous variables reduces to an LP, which can be solved efficiently in polynomial time. Consequently, the main computational challenge in MILPs lies in finding high-quality assignments for the discrete variables. We therefore focus on optimizing the discrete variables and adopt a decomposition perspective inspired by Benders de-

composition, which we apply to Equation 7:

$$\begin{aligned} & \min_{\mathbf{x} \in \mathcal{X}} \mathbf{c}^\top \mathbf{x} + f(\mathbf{x}) \\ \text{where } f(\mathbf{x}) &= \max_{\mathbf{u} \geq 0, \mathbf{u} \in \mathbb{R}^m} \{(\mathbf{b} - \mathbf{A}\mathbf{x})^\top \mathbf{u} | \mathbf{B}^\top \mathbf{u} \leq \mathbf{d}\} \quad (11) \\ &= - \min_{\mathbf{u} \geq 0, \mathbf{u} \in \mathbb{R}^m} \{(\mathbf{A}\mathbf{x} - \mathbf{b})^\top \mathbf{u} | \mathbf{B}^\top \mathbf{u} \leq \mathbf{d}\} \end{aligned}$$

If Equation 11 was solved using exact MILP solvers, the resulting algorithm would correspond to the classical Benders decomposition. Our approach exploits only the decomposition structure induced by this formulation and avoids explicitly solving CO problems. Instead, we resort to gradient-based methods to search for improved integer assignments.

This reformulation yields a simplified objective that permits gradient-based updates with respect to the discrete variables, i.e., $\mathbf{c} + \nabla_{\mathbf{x}} f(\mathbf{x})$. The challenge that remains lies in how to derive $\nabla_{\mathbf{x}} f(\mathbf{x})$, where $f(\mathbf{x})$ is defined as the optimal value of a parameterized LP, which doesn't seem differentiable.

4.2. Gradient Calculation via Envelope Theorem

Since the derivative is taken on the objective of this parametrized LP, we don't need to calculate according to KKT conditions as in OptNet (Amos & Kolter, 2017). Instead, we refer to the envelope theorem (Milgrom & Segal, 2002), a major result about the differentiability properties of the value function of a parameterized optimization problem. Consider the following constrained problem:

$$\min h(\mathbf{x}, \mathbf{u}), \quad \text{s.t. } g(\mathbf{x}, \mathbf{u}) \geq 0 \quad (12)$$

where $\mathbf{x} \in \mathbb{R}^p$ are the parameters of interest. The Lagrangian of this problem is given by:

$$\mathcal{L}(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = h(\mathbf{x}, \mathbf{u}) + \boldsymbol{\lambda}^\top g(\mathbf{x}, \mathbf{u}) \quad (13)$$

If we denote $\mathbf{u}^*(\mathbf{x})$ as the optimal solution to Eq. 12, define $\mathcal{L}^*(\mathbf{x}) = \mathcal{L}(\mathbf{x}, \mathbf{u}^*(\mathbf{x}), \boldsymbol{\lambda}^*(\mathbf{x}))$ and name $V(\mathbf{x}) = h(\mathbf{x}, \mathbf{u}^*(\mathbf{x}))$ as the value function, we have the following envelope theorem:

Theorem 4.1. *Assume that V and $\mathcal{L}$ are continuously differentiable. Then we have*

$$\frac{\partial V}{\partial \mathbf{x}_i} = \frac{\partial \mathcal{L}^*}{\partial \mathbf{x}_i} = \frac{\partial \mathcal{L}(\mathbf{x}, \mathbf{u}^*(\mathbf{x}), \boldsymbol{\lambda}^*(\mathbf{x}))}{\partial \mathbf{x}_i}, i = 1, 2, \dots, p \quad (14)$$

where $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_i} = \frac{\partial h}{\partial \mathbf{x}_i} + \boldsymbol{\lambda}^\top \frac{\partial g}{\partial \mathbf{x}_i}$.

In Eq. 9, the parameters $\mathbf{x}$ appear only in the objective h . Hence, the gradient can be simplified to the gradient of the objective function with respect to the integer variables $\mathbf{x}$ evaluated at the optimal solution of the subproblem represented by Eq. 9. If we denote the solution to the dual problem by $\mathbf{u}^*$, the gradient can be represented as:

$$\frac{\partial f}{\partial \mathbf{x}} = -\mathbf{A}^\top \mathbf{u}^* \quad (15)$$

Hence, given any $\mathbf{x}$, the gradient of the objective value with respect to the integer variable $\mathbf{x}$ is $\mathbf{d} - \mathbf{A}^\top \mathbf{u}^*$. It is noteworthy that $\mathbf{x}$ is $\mathbf{d} - \mathbf{A}^\top \mathbf{u}^*$, which is actually a subgradient, since the dual optimum $\mathbf{u}^*$ may not be unique. In practice, commercial solvers such as Gurobi typically return a basic optimal solution, corresponding to an extreme point of the dual feasible region. These extreme subgradients are often sparse, yielding sharper update directions that enable the sampler to explore the integer space more aggressively.

For linear programs whose dual problem is feasible and bounded, strong duality holds, implying that the primal objective value can be recovered by solving only the dual problem. Moreover, the above result shows that no additional gradient computation is required once the optimal dual solution is obtained: solving a single LP suffices for both the forward objective evaluation and the backward subgradient computation.

Although differentiable LP solvers such as CVXPY (Diamond & Boyd, 2016) could in principle be used to enable end-to-end differentiation, leveraging the closed-form subgradient in Eq. 15 allows us to exploit the efficiency and numerical robustness of commercial solvers such as Gurobi (Gurobi Optimization, LLC, 2024). This advantage is particularly critical for large-scale LPs, where solver efficiency directly impacts overall training and sampling performance.

4.3. Handling Infeasible Integer Solution Guess

All of the above formulations are based on the fact that the remaining LP, i.e., Eq. 9 is bounded and feasible. This cannot be guaranteed by this decomposition. In the original Benders decomposition, if the integer solution guess $\bar{\mathbf{x}}$ provided by the master problem is infeasible for the primal subproblem in Eq. 8, which corresponds to unboundedness for Eq. 9, a feasibility cut will be added to rule this solution out of the master problem. Similarly, we should find a direction that reduces the extent of constraint violation when faced with an infeasible $\bar{\mathbf{x}}$.

When we solve Eq. 9 with commercial solvers such as Gurobi and the unboundedness is detected, an extreme ray $\mathbf{u}$ can also be found, which provides a certificate that this problem is unbounded. In another way, this ray satisfies that $\mathbf{B}^\top \mathbf{u} \leq \mathbf{d}, \mathbf{u} \geq 0, (\mathbf{b} - \mathbf{A}\bar{\mathbf{x}})^\top \mathbf{u} \geq 0$, thereby providing a valid proof that the subproblem is unbounded. In classical Benders decomposition, such a ray would be used to generate a feasibility cut to exclude the current integer assignment from the master problem. In contrast, rather than explicitly solving the resulting integer program, we incorporate this information into our neural framework via gradient-based updates. What we need is a solid and helpful direction to guide the solution out of this infeasible area. We would, in

turn, solve the following problem to find the best direction:

$$\min_{\mathbf{y} \geq 0, \mathbf{s} \geq 0} \mathbf{1}^\top \mathbf{s}, \quad \text{s.t.} \quad \mathbf{B}\mathbf{y} + \mathbf{s} \geq \mathbf{b} - \mathbf{A}\bar{\mathbf{x}} \quad (16)$$

where $\mathbf{1}$ is an all-one vector. By solving the above problem, we find a $\mathbf{y}$ that provides the tightest certificate. This is because we require the minimum outside intervention, which is indicated by the slack variables $\mathbf{s}$. The dual of Eq. 16 is:

$$\max_{\mathbf{u}} (\mathbf{b} - \mathbf{A}\bar{\mathbf{x}})^\top \mathbf{u}, \quad \text{s.t.} \quad \mathbf{B}^\top \mathbf{u} \leq \mathbf{0}, \quad 0 \leq \mathbf{u} \leq \mathbf{1} \quad (17)$$

Through a similar proof for the feasible case, we find that the gradient that pushes $\mathbf{x}$ away from this infeasible guess is $-\mathbf{A}^\top \mathbf{u}^*$, where $\mathbf{u}^*$ is the optimum solution to Eq. 17.

In addition to directly providing gradients for optimization, we will also provide a value back for objective value evaluation. When $\mathbf{x}$ yields a feasible subproblem, the objective value given $\mathbf{x}$ is set as the energy function. When an infeasible $\mathbf{x}$ is encountered, we will first set a large enough value, such as 10^6 , to separate this from a feasible $\mathbf{x}$. Furthermore, we add $\mathbf{1}^\top \mathbf{s}$ to this large value to show how far this $\mathbf{x}$ is from the feasible space. When gradients are not utilized (as discussed in the next section), $\mathbf{1}^\top \mathbf{s}$ will serve as an indicator of the relative quality of those infeasible assignments.

4.4. Neural Langevin Sampler

In the previous sections, we formulated how to evaluate the quality of integer assignment $\mathbf{x}$. As discussed in section 3, solving a combinatorial optimization problem is equivalent to sampling from an energy-based model, which can be realized via Langevin dynamics (LD). In this section, we will discuss in detail how LD might be of help in our framework.

An ideal sampling distribution $q(\mathbf{x}'|\mathbf{x})$ is defined as in Eq. 5. Note that the score function of the energy-based method is:

$$s_\tau(\mathbf{x}) = \nabla \log p_\tau(\mathbf{x}) = -\frac{1}{\tau} \nabla H(\mathbf{x}) \quad (18)$$

As defined in Eq. 2, $H(\mathbf{x})$ is composed of the objective value and constraint violation. For feasible $\mathbf{x}$, constraint violation is 0. The energy function equals the objective value. For infeasible $\mathbf{x}$, we will set the objective value to a large value, and the extent of violation is calculated as in the previous section.

Now that we have defined the ideal transition probability, the next question is how to learn it. We seek help from a Langevin neural network proposed in (Feng & Yang, 2025). The model is trained to minimize the following objective:

$$L = \mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[H(\mathbf{x}')] \quad (19)$$

However, it is impossible to calculate the energy for all $\mathbf{x}'$ since we need to solve infinitely many LP instances. We propose three alternative loss functions for learning.

REINFORCE loss. Eq. 19 minimizes the conditional expectation of the energy function after the one-step update. One natural way to optimize this loss may be to estimate the policy gradient, i.e. $\mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[H(\mathbf{x}') \nabla \log q_\theta(\mathbf{x}'|\mathbf{x})]$ using Monte Carlo sampling. This REINFORCE loss does not require the reward (i.e., the energy function) to be differentiable and is therefore broadly applicable. However, this generality comes at the cost of efficiency, as the gradient estimator captures limited structural information and typically exhibits high variance. Notably, this term is also the only component of the objective directly affected by the choice of a large penalty coefficient for infeasible solutions.

Erdős loss. Secondly, we observe that if the expectation is tractable, this term is equivalent to the unsupervised learning loss in (Karalias & Loukas, 2020), which measures the energy of an expected solution. At first glance, computing $H(\mathbf{x})$ appears intractable due to the presence of a subproblem. However, we can show that it is possible to efficiently obtain a lower bound on this conditional expectation, providing a practical surrogate for optimization.

Theorem 4.2. $H(\mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[\mathbf{x}']) \leq \mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[H(\mathbf{x}')]$

To make $H(\mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[\mathbf{x}'])$ a better approximation of the original goal Eq. 19, we introduce a second order correction term. We denote $\boldsymbol{\mu} = \mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[\mathbf{x}']$. According to the second-order Taylor expansion, we have:

$$\mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[H(\mathbf{x}')] \approx H(\boldsymbol{\mu}) + \frac{1}{2} \text{Tr}(\nabla^2 H(\boldsymbol{\mu}) \text{Cor}(\mathbf{x}')) \quad (20)$$

Since H is a piece-wise linear function whose curvature comes only from the basis changes of linear programming problems, we do not have a closed-form formulation of $\nabla^2 H$. However, we could approximate it using the gradient we just calculated (Wang, 2012), i.e.,

$$\nabla^2 H(\boldsymbol{\mu}) \approx \nabla H(\boldsymbol{\mu}) \nabla H(\boldsymbol{\mu})^\top \quad (21)$$

which is a Gauss-Newton approximation. This effectively captures the empirical sensitivity of the objective to variable assignments, providing a stable surrogate for the second-order correction term in the Taylor expansion. Furthermore, when given the current solution $\mathbf{x}$, each variable in $\mathbf{x}'$ flips independently. Hence, $\text{Cor}(\mathbf{x}') = \text{diag}(\boldsymbol{\mu} \odot (1 - \boldsymbol{\mu}))$. Using the property of the trace, we can ultimately calculate:

$$\mathbb{E}[H(\mathbf{x}')] \approx H(\boldsymbol{\mu}) + \frac{1}{2} \nabla H(\boldsymbol{\mu})^\top \text{diag}(\boldsymbol{\mu} \odot (1 - \boldsymbol{\mu})) \nabla H(\boldsymbol{\mu}) \quad (22)$$

Experimentally, we only consider gradients related to continuous variables in $\nabla H(\mathbf{x})$. This choice is motivated by the observation that any curvature in H arises solely from changes in the LP basis corresponding to the continuous variables. Ignoring gradients with respect to the integer variables often yields a more accurate approximation in practice. However, this approximation does not provide

Table 1. Experimental results on two capacitated facility location datasets. ‘+LP’ means that we solve a corresponding LP for continuous variables, given solutions generated by the aforementioned neural solvers.

Method	CFLP-(100, 100)				CFLP-(200, 200)			
	Obj ↓	Gap ↓	Time ↓	Feas ↑	Obj ↓	Gap ↓	Time ↓	Feas ↑
Gurobi (Gurobi Optimization, LLC, 2024)	17876.29	-	7.44s	100%	33922.88	-	31.91s	100%
SL + LP	18731.74	4.78%	5.28s	100%	35897.17	5.82%	30.12s	100%
DIFUSCO (Sun & Yang, 2023a) + LP	18502.39	3.50%	10.48s	100%	36932.23	8.87%	52.21s	100%
FMIP (Li et al., 2026)	NaN	NaN	5.07s	0%	NaN	NaN	46.79s	0%
FMIP + LP	22368.19	25.10%	7.84s	75.0%	35843.53	5.66%	47.79s	31.7%
DiffILO (Geng et al., 2025)	NaN	NaN	0.05s	0%	NaN	NaN	0.10s	0%
DiffILO + LP	NaN	NaN	0.14s	0%	NaN	NaN	5.25s	0%
Ours (Erdős)	18992.49	6.24%	4.73s	100%	37981.29	11.96%	28.12s	100%
Ours (Gumbel)	21830.38	22.12%	4.89s	100%	40130.28	18.74%	29.81s	100%
Ours (REINFORCE)	22190.32	24.13%	4.79s	100%	40921.49	20.63%	29.97s	100%

a strict upper bound on the target objective, and as a result, performance on certain datasets may be suboptimal. By contrast, a REINFORCE-based loss, while potentially higher-variance, offers a more theoretically principled alternative with stronger guarantees of correctness.

Gumbel loss. A third option is to use Gumbel-Softmax for differentiable sampling. This approach combines the advantages of gradient-based optimization with the ability to handle discrete variables, producing more accurate update directions. Both the Gumbel loss and the REINFORCE loss suffer from high variance due to sampling, but differ in how sampling is performed and how gradients are computed. In particular, the Gumbel-Softmax loss better exploits the differentiability provided by the environment.

Following Feng & Yang (2025), we train the Langevin neural network using a reinforcement-learning-style approach, considering only the immediate next state. This avoids the high variance that arises when estimating long-term returns. Specifically, we sequentially draw T samples from the current proposal distribution $q(\mathbf{x}' | \mathbf{x})$, and for each sample, we update the network to minimize the aforementioned loss.

5. Experiment

5.1. Experimental Setup

Datasets & Benchmarks. We evaluate our model on three widely studied MILPs, i.e., capacitated facility location problem (CFLP) (Feng et al., 2025), item placement (IP), and load balancing (LB) (Gasse et al., 2022). The CFLP datasets are generated following (Feng et al., 2025). The instances for IP and LB are directly adopted from (Feng et al., 2025). We generate two CFLP datasets, with 100 facilities, 100 customers, and 200 facilities and 200 customers, respectively. All datasets have 1000 train instances, 100 valid, and 100 test instances. The reference labels are generated via Gurobi (Gurobi Optimization, LLC, 2024),

with a time limit of 3600s. More details are provided in Appendix B.

Baselines. We compare our method against four representative learning-based MILP solvers and Gurobi: 1) supervised learning (SL), where models are trained to directly predict solutions from problem instances. In this setting, we restrict the prediction to integer variables and optimize a cross-entropy loss; 2) DIFUSCO (Sun & Yang, 2023a), which employs a diffusion model to generate integer solutions for MILPs; 3) FMIP (Li et al., 2026): a joint continuous-discrete flow-matching framework that simultaneously predicts integer and continuous variables; 4) DiffILO (Geng et al., 2025): a state-of-the-art unsupervised training framework originally designed for integer linear programming problems. Since the relaxation and sampling of integer variables mentioned in DiffILO do not interfere with continuous variables, the introduction of continuous variables is direct.

All neural baselines employ a special graph convolution network designed for bi-partite graphs (Li et al., 2026) to extract features from instances. Implementation details and architectural specifications are provided in Appendix B.3.

Downstream solvers postprocessing. For complicated MILPs, the output of a neural solver is usually of low quality. Instead of directly employing those solutions, they usually serve as a warm start for downstream solvers, enabling them to search for better solutions more efficiently. It is noteworthy that those downstream solvers only take integer predictions. Since downstream solvers tackle a more restricted version of the problems, either a shorter solving time or better solution quality can be achieved. Although indirectly, the solution quality produced by downstream solvers can effectively reflect the quality of the warm-start solution.

For the specific choice of downstream solvers, we select Neural Diving (ND) (Nair et al., 2021), Predict-and-Search

Table 2. Experimental results on the item placement and load balancing dataset.

Downstream Solver	Method	IP			LB		
		Obj ↓	Gap ↓	Time ↓	Obj ↓	Gap ↓	Time ↓
-	Gurobi (Gurobi Optimization, LLC, 2024)	17.73	-	3600s	706.00	-	3600s
ND (400s)	SL	14.62	-17.54%	400s + 1.69s	719.67	1.94%	400s + 24.49s
	DIFUSCO (Sun & Yang, 2023a)	15.41	13.08%	400s + 2.03s	717.73	1.66%	400s + 29.40s
	FMIP (Li et al., 2026)	13.99	-21.09%	400s + 2.17s	706.30	0.04%	400s + 29.88s
	Ours (Erdős)	13.89	-21.65%	400s + 1.84s	711.92	0.84%	400s + 24.62s
	Ours (Gumbel)	13.82	-22.05%	400s + 1.84s	706.10	0.01%	400s + 24.62s
	Ours (REINFORCE)	14.11	-18.55%	400s + 1.84s	709.67	0.51%	400s + 24.62s
PS (600s)	SL	15.34	-13.48%	600s + 1.69s	749.60	6.17%	600s + 24.49s
	DIFUSCO (Sun & Yang, 2023a)	14.73	-16.92%	600s + 2.03s	714.11	1.15%	600s + 29.40s
	FMIP (Li et al., 2026)	13.92	-21.48%	600s + 2.17s	705.90	-0.01%	600s + 29.88s
	Ours (Erdős)	13.69	-22.78%	600s + 1.84s	713.21	1.02%	600s + 24.62s
	Ours (Gumbel)	13.77	-22.33%	600s + 1.84s	705.90	-0.01%	372s + 24.62s
	Ours (REINFORCE)	13.99	-21.09%	600s + 1.84s	706.00	0.00%	600s + 24.62s
Apollo (800s)	SL	14.16	-20.13%	800s + 1.69s	706.00	0.00%	800s + 24.49s
	DIFUSCO (Sun & Yang, 2023a)	14.01	-20.98%	800s + 2.03s	709.32	0.47%	800s + 29.40s
	FMIP (Li et al., 2026)	13.74	-22.50%	800s + 2.17s	705.90	-0.01%	800s + 29.88s
	Ours (Erdős)	13.62	-23.18%	800s + 1.84s	707.00	0.14%	800s + 24.62s
	Ours (Gumbel)	13.64	-23.07%	800s + 1.84s	705.90	-0.01%	800s + 24.62s
	Ours (REINFORCE)	13.77	-22.33%	800s + 1.84s	706.00	0.00%	800s + 24.62s
None	SL	545.57	2977%	1.89s	NaN	NaN	26.29s
	DIFUSCO (Sun & Yang, 2023a)	677.30	3718%	2.87s	NaN	NaN	36.28s
	FMIP (Li et al., 2026) + LP	637.14	3493%	3.37s	NaN	NaN	33.48s
	Ours (Erdős)	483.14	2624%	1.84s	NaN	NaN	24.62s
	Ours (Gumbel)	481.39	2613%	1.84s	872.00	23.51%	24.62s
	Ours (REINFORCE)	499.29	2716%	1.84s	891.00	26.20%	24.62s

(PS) (Han et al., 2023), and Apollo-MILP (Apollo) (Liu et al., 2025). Downstream solvers are only applied to IP and LB datasets due to their complexity. For simpler datasets such as CFLP datasets, the only post-processing operation we do is to generate full solutions by solving LPs.

Metrics. We adopted four metrics to better evaluate the performance of different methods. 1) Objective value (Obj): the objective value given the predicted solutions. For CFLP, we only calculate objective values given those feasible ones. 2) Gap: the objective gap, calculated as $\frac{\text{obj}-\text{baseline}}{|\text{baseline}|}$, where the baselines are the objective values solved by Gurobi. These baseline results might be suboptimal due to the time limit that we impose on Gurobi. Hence, the gap might be negative in some scenarios. 3) Inference time per instance (Time): the inference time for neural solvers. For solvers with downstream solvers, the solving time for downstream solvers is also included. 4) Feasibility (Feas): For CFLP, we also evaluated the feasibility of the generated solutions. For IP and LB datasets, the feasibility is either 0% or 100%, which can be inferred from the presence of NaN.

5.2. Main Results

In this section, we analyze the experimental results on those four datasets. Table 1 reports our results on two CFLP

datasets. The results indicate that supervised learning (SL) achieved the best performance among all methods. All neural solvers outperform traditional solvers in speed. However, FMIP fails to capture feasibility on both CFLP datasets. To provide a fair comparison, we additionally evaluate a variant in which the integer variables predicted by FMIP are fixed, and the remaining continuous variables are recovered by solving the corresponding LP. While this post-processing procedure improves feasibility, it still fails to guarantee fully feasible solutions, suggesting that directly extending ILP-oriented generative models to the mixed-integer setting remains challenging. A similar phenomenon is observed for DiffILO. Even with LP-based post-processing, DiffILO consistently converges to trivial all-zero solutions across all instances, regardless of the penalty coefficient used during optimization.

These observations indicate that, although extending existing ILP methodologies to MILPs appears conceptually straightforward, effectively modeling the interaction between integer and continuous variables requires additional algorithmic considerations. In contrast, the proposed framework consistently produces feasible solutions and achieves the strongest performance among the unsupervised approaches evaluated on these benchmarks.

Table 3. Experimental results on CFLP with and without the correction term, where w. represents results with the proposed correction term and wo. for the ones without. w2. refers to experiments done with an L2 regularization.

Method	CFLP-(100, 100)				CFLP-(200, 200)			
	Obj ↓	Gap ↓	Time ↓	Feas ↑	Obj ↓	Gap ↓	Time ↓	Feas ↑
Ours (Erdős) w.	19117.49	6.94%	4.83s	100%	37998.29	12.01%	28.12s	100%
Ours (Erdős) wo.	20983.15	17.40%	4.83s	100%	39938.12	17.73%	28.12s	100%
Ours (Erdős) w2.	20217.15	13.09%	4.83s	100%	39029.12	15.05%	28.12s	100%

Among the three proposed training objectives, the Erdős loss delivers the best overall performance on the CFLP datasets. Although our inference procedure still incorporates LP solves to select the best solutions along generated trajectories, the resulting runtime remains highly competitive. While a performance gap remains between our method and fully supervised models, it is important to emphasize that our framework operates entirely without supervision. From this perspective, the proposed approach shows strong effectiveness and represents a competitive unsupervised alternative to solve MILPs.

Table 2 shows the results of our methods on the item placement and load balancing datasets. On the IP benchmark, our method achieves the best overall performance. For both datasets, the Gumbel loss consistently outperforms the other training objectives, which may reflect differences in the problem structure compared to the CFLP instances. Another noteworthy observation is that our approach is the only method capable of producing feasible solutions before any post-processing. Although the resulting objective gaps remain larger than those obtained after refinement, the corresponding inference times are substantially shorter. Consequently, these raw solutions can serve as effective approximations in scenarios where computational efficiency is the primary concern.

A key distinction between the CFLP datasets and the IP/LB benchmarks lies in the quality of the available labels. The CFLP instances are annotated with optimal solutions, whereas the IP and LB datasets contain only suboptimal labels due to the computational complexity of obtaining exact optima. Importantly, our unsupervised training framework does not rely on optimal supervision. As a result, its performance is expected to be less sensitive to label quality and potentially even more advantageous in large-scale real-world applications, where optimal solutions are typically unavailable or prohibitively expensive to obtain.

6. Ablation Studies

To assess the contribution of the proposed correction term, we conducted an ablation study to evaluate its effect on solution quality. Specifically, we compare models trained with and without the correction term applied to the Erdős

loss on the CFLP dataset. Furthermore, we compare an additional baseline regularizer of the form $\mathbb{E}[\mathbf{x}'](1 - \mathbb{E}[\mathbf{x}'])$, which encourages the expected binary decision variables to approach either 0 or 1. As $\mathbb{E}[\mathbf{x}']$ becomes closer to these extremes, the surrogate loss more accurately reflects the underlying combinatorial objective. We refer to this baseline as an ℓ_2 -style regularization term.

The results are reported in Table 3. Incorporating the proposed correction term consistently yields substantial performance improvements over both the unregularized variant and the baseline regularizer. In addition, the resulting solutions exhibit a higher overall quality, highlighting the effectiveness of the proposed correction mechanism.

7. Conclusion

In this work, we introduce a novel unsupervised neural Langevin sampler for solving general mixed integer linear programs. Our solver employs a hybrid architecture, where neural networks predict integer variables while the corresponding continuous variables are handled by solving the remaining linear program. A Langevin dynamics-based neural sampler learns to iteratively refine integer assignments under the guidance of an energy function that takes care of both the objective value and the constraint violation. By leveraging the envelope theorem, the entire pipeline remains differentiable, enabling end-to-end gradient-based optimization. Experiments on three representative large-scale MILPs show that our method manages to consistently generate 100% feasible solutions without expensive post-processing methods. The comparison with supervised solvers further demonstrates the superiority of our method.

Limitations. In this work, only MILP with binary variables is considered. Though extensions are natural, due to the complexity, we will leave this part for future work.

Impact Statement

This paper presents work whose goal is to solve mixed integer linear programs using machine learning techniques. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Ahuja, R. K., Ergun, Ö., Orlin, J. B., and Punnen, A. P. A survey of very large-scale neighborhood search techniques. *Discrete Applied Mathematics*, 123(1-3):75–102, 2002.
- Amos, B. and Kolter, J. Z. Optnet: differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ICML’17, pp. 136–145. JMLR.org, 2017.
- Ceria, S., Cordier, C., Marchand, H., and Wolsey, L. A. Cutting planes for integer programs with general integer variables. *Mathematical programming*, 81:201–214, 1998.
- Charikar, M. and Guha, S. Improved combinatorial algorithms for the facility location and k-median problems. In *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*, pp. 378–388, 1999. doi: 10.1109/SFFCS.1999.814609.
- Delarue, A., Anderson, R., and Tjandraatmadja, C. Reinforcement learning with combinatorial actions: An application to vehicle routing. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 609–620. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/06a9d51e04213572ef0720dd27a84792-Paper.pdf.
- Diamond, S. and Boyd, S. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 2016. URL https://stanford.edu/~boyd/papers/pdf/cvxpy_paper.pdf. To appear.
- Feng, S. and Yang, Y. Regularized langevin dynamics for combinatorial optimization. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=bbJ0QCujU4>.
- Feng, S., Sun, W., Li, S., Talwalkar, A., and Yang, Y. A comprehensive evaluation of contemporary machine-learning-based solvers for CO. In *2nd AI for Math Workshop @ ICML 2025*, 2025. URL <https://openreview.net/forum?id=BChsy6cNb1>.
- Feng, S., Kim, J., Yang, Y., Boudreau, J., Chowdhury, T., Hoisie, A., Khan, R., Kilic, O. O., Klasky, S., Korchuganova, T., Nilsson, P., Martinez Outschoorn, V. I., Park, D. K., Podhorszki, N., Ren, Y., Suter, F., Sri Vatsavai, S., Yang, W., Yoo, S., Maeno, T., and Klimentov, A. Alternative mixed integer linear programming optimization for joint job scheduling and data allocation in grid computing. *Future Generation Computer Systems*, 175:108075, 2026. ISSN 0167-739X. doi: <https://doi.org/10.1016/j.future.2025.108075>. URL <https://www.sciencedirect.com/science/article/pii/S0167739X25003693>.
- Gasse, M., Chetelat, D., Ferroni, N., Charlin, L., and Lodi, A. Exact combinatorial optimization with graph convolutional neural networks. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/d14c2267d848abeb81fd590f371d39bd-Paper.pdf.
- Gasse, M., Bowly, S., Cappart, Q., Charfreitag, J., Charlin, L., Chételat, D., Chmiela, A., Dumouchelle, J., Gleixner, A., Kazachkov, A. M., Khalil, E., Lichocki, P., Lodi, A., Lubin, M., Maddison, C. J., Christopher, M., Papageorgiou, D. J., Parjadis, A., Pokutta, S., Prouvost, A., Scavuzzo, L., Zarpellon, G., Yang, L., Lai, S., Wang, A., Luo, X., Zhou, X., Huang, H., Shao, S., Zhu, Y., Zhang, D., Quan, T., Cao, Z., Xu, Y., Huang, Z., Zhou, S., Binbin, C., Minggui, H., Hao, H., Zhiyu, Z., Zhiwu, A., and Kun, M. The machine learning for combinatorial optimization competition (ml4co): Results and insights. In Kiela, D., Ciccone, M., and Caputo, B. (eds.), *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, volume 176 of *Proceedings of Machine Learning Research*, pp. 220–231. PMLR, 06–14 Dec 2022. URL <https://proceedings.mlr.press/v176/gasse22a.html>.
- Geng, Z., Wang, J., Li, X., Zhu, F., HAO, J., Li, B., and Wu, F. Differentiable integer linear programming. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=FPfCUJTsCn>.
- Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2024. URL <https://www.gurobi.com>.
- Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. A GNN-guided predict-and-search framework for mixed-integer linear programming. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=pHMpgT5xWaE>.
- Karalias, N. and Loukas, A. Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In Larochelle, H., Ranzato,

- M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 6659–6672. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/49f85a9ed090b20c8bed85a5923c669f-Paper.pdf.
- Ku, W.-Y. and Beck, J. C. Mixed integer programming models for job shop scheduling: A computational analysis. *Computers Operations Research*, 73:165–173, 2016. ISSN 0305-0548. doi: <https://doi.org/10.1016/j.cor.2016.04.006>. URL <https://www.sciencedirect.com/science/article/pii/S0305054816300764>.
- Labassi, A. G., Ch  telat, D., and Lodi, A. Learning to compare nodes in branch and bound with graph neural networks. *Advances in neural information processing systems*, 35:32000–32010, 2022.
- LeCun, Y., Chopra, S., Hadsell, R., Ranzato, M., Huang, F., et al. A tutorial on energy-based learning. *Predicting structured data*, 1(0), 2006.
- Li, H., Yuan, H., Zhang, H., Lin, J., Ge, D., Wang, M., and Ye, Y. FMIP: Joint continuous-integer flow for mixed-integer linear programming. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=kyvW6S0u3z>.
- Liu, H., Wang, J., Geng, Z., Li, X., Zong, Y., Zhu, F., HAO, J., and Wu, F. Apollo-MILP: An alternating prediction-correction neural solving framework for mixed-integer linear programming. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=mFY0tPDWK8>.
- Milgrom, P. and Segal, I. Envelope theorems for arbitrary choice sets. *Econometrica*, 70(2):583–601, 2002.
- Nair, V., Bartunov, S., Gimeno, F., von Glehn, I., Lichocki, P., Lobov, I., O’Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., Addanki, R., Hapuarachchi, T., Keck, T., Keeling, J., Kohli, P., Ktena, I., Li, Y., Vinyals, O., and Zwols, Y. Solving mixed integer programs using neural networks, 2021. URL <https://arxiv.org/abs/2012.13349>.
- Papadimitriou, C. and Steiglitz, K. *Combinatorial Optimization: Algorithms and Complexity*, volume 32. 01 1982. ISBN 0-13-152462-3. doi: 10.1109/TASSP.1984.1164450.
- Puigdemont, P., Skoulakis, S., Chrysos, G. G., and Cevher, V. Learning to remove cuts in integer linear programming. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Rahmaniani, R., Crainic, T. G., Gendreau, M., and Rei, W. The benders decomposition algorithm: A literature review. *European Journal of Operational Research*, 259 (3):801–817, 2017.
- Sanokowski, S., Hochreiter, S., and Lehner, S. A diffusion model framework for unsupervised neural combinatorial optimization. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=AFfXlKFHXJ>.
- Scavuzzo, L., Aardal, K., Lodi, A., and Yorke-Smith, N. Machine learning augmented branch and bound for mixed integer linear programming. *Mathematical Programming*, pp. 1–44, 2024.
- Sun, Z. and Yang, Y. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023a. URL <https://openreview.net/forum?id=JV8Ff0lgVV>.
- Sun, Z. and Yang, Y. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023b. URL <https://openreview.net/forum?id=JV8Ff0lgVV>.
- Wang, Y. Gauss–newton method. *WIREs Comput. Stat.*, 4(4):415–420, June 2012. ISSN 1939-5108. doi: 10.1002/wics.1202. URL <https://doi.org/10.1002/wics.1202>.
- Welling, M. and Teh, Y. W. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th International Conference on International Conference on Machine Learning, ICML’11*, pp. 681–688, Madison, WI, USA, 2011. Omnipress. ISBN 9781450306195.
- Wolsey, L. A. *Integer programming*. John Wiley & Sons, 2020.
- Wolsey, L. A. and Nemhauser, G. L. *Integer and combinatorial optimization*. John Wiley & Sons, 1999.
- Zhang, J., Liu, C., Li, X., Zhen, H.-L., Yuan, M., Li, Y., and Yan, J. A survey for solving mixed integer programming via machine learning. *Neurocomputing*, 519:205–217, 2023.
- Zhang, R., Liu, X., and Liu, Q. A langevin-like sampler for discrete distributions. *International Conference on Machine Learning*, 2022.

A. Proof

Proof of Theorem 4.2. Firstly, as from Eq. 11, we have:

$$H(\mathbf{x}) = \mathbf{c}^\top \mathbf{x} - \min_{\mathbf{u}} \{ -(\mathbf{b} - \mathbf{A}\mathbf{x})^\top \mathbf{u} \mid \mathbf{B}^\top \mathbf{u} \leq \mathbf{d}, \mathbf{u} \geq 0, \mathbf{u} \in \mathbb{R}^m \} \quad (23)$$

We would love to use the property that the point-wise maximum of a convex/linear function is still a convex function. Hence, we can prove that $H(\mathbf{x})$ is a convex function.

Secondly, we apply Jensen's inequality, and we have: $H(\mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[\mathbf{x}']) \leq \mathbb{E}_{q_\theta(\mathbf{x}'|\mathbf{x})}[H(\mathbf{x}')] \quad \square$

B. Experiment Details

B.1. Experiment Settings

In this section, we provide details regarding our experiment settings. All models are trained with trajectories of 10 steps and evaluated by sampling a 10-step trajectory. In each iteration, we sample as many trajectories as possible, depending on the size of the training data. All experiments are run on 8 RTX A6000 GPUs.

B.2. Datasets

In this section, we provide the formulation of each dataset and the additional experiment design for each dataset.

B.2.1. CAPACITATED FACILITY LOCATION PROBLEM

The capacitated facility location problem (CFLP) is the basis for many real-life optimization problems, where we aim to find optimal sites to build facilities to satisfy customers' demands under the constraints that there is a limit on the total demand that each facility can satisfy. Consider that there are M customers and N possible sites for facilities. Continuous variables $y_{ij} \geq 0$ represent the amount of services provided by the facility j to the customer i . The binary variables x_j indicate whether the site is in use. The CFLP problem can be formulated as follows:

$$\begin{aligned} \min \quad & \sum_{j=1}^N f_j x_j + \sum_{i=1}^M \sum_{j=1}^N c_{ij} y_{ij} \\ \text{s.t.} \quad & \sum_{j=1}^N y_{ij} = d_i && \text{for } i = 1, \dots, M \\ & \sum_{i=1}^M y_{ij} \leq M_j x_j && \text{for } j = 1, \dots, N \\ & y_{ij} \leq d_i x_j, && \text{for } i = 1, \dots, M; j = 1, \dots, N \\ & y_{ij} \geq 0, && \text{for } i = 1, \dots, M; j = 1, \dots, N \\ & x_j \in \{0, 1\}, && \text{for } j = 1, \dots, N \end{aligned} \quad (24)$$

If we compare this formulation with the standard formulation as in Eq. 7, we can see that $p = N, n = M \times N + N$. Hence, if we use CFLP-(100, 100), this dataset has 100 integer variables and 10000 continuous variables. Furthermore, to make this compatible with the standard formulation, we split the equality constraint into two inequality constraints.

B.2.2. ITEM PLACEMENT

Item placement (IP) describes the problem of assigning discrete items to containers in an evenly distributed way. Suppose that there are M items and N containers, and that we operate in a 3-dimensional space. This problem can be represented by the following MILP. Though z_k seems unbounded in this representation, we can see that $z_k \geq y_{jk} \geq 0$. If we align this formulation with the standard formulation, we can observe that $p = MN, n = MN + 3N + 3$. The official item placement dataset (Gasse et al., 2022) sets $M = 105$ and $N = 10$.

It is noteworthy that some constraints depend solely on integer variables, i.e., $\sum_{j=1}^b x_{ij} = 1$. It leaves no space for optimization over continuous variables. To enable gradient-based optimization, we apply the Gumbel-Softmax trick,

ensuring that the sampled solutions satisfy the constraints during training. When sampling for training trajectories, this Gumbel Softmax is also imposed to improve the quality of the sampled trajectories. Experimental results demonstrate that this simple modification leads to a significant improvement in our model’s performance.

$$\begin{aligned}
 & \min \sum_{j=1}^N \sum_{k=1}^3 \alpha_k y_{jk} + \sum_{k=1}^3 \beta_k z_k \\
 & \text{s.t. } \sum_{j=1}^N x_{ij} = 1, & \text{for } i = 1, \dots, M \\
 & \sum_{i=1}^M c_{ik} x_{ij} \leq d_k, & \text{for } j = 1, \dots, N; k = 1, 2, 3 \\
 & \sum_{i=1}^M f_{ik} x_{ij} + y_{jk} \geq 1, & \text{for } j = 1, \dots, N; k = 1, 2, 3 \\
 & y_{jk} \leq z_k, & \text{for } j = 1, \dots, N; k = 1, 2, 3 \\
 & x_{ij} \in \{0, 1\}, & \text{for } i = 1, \dots, M; j = 1, \dots, N \\
 & y_{jk} \geq 0, & \text{for } j = 1, \dots, N; k = 1, 2, 3
 \end{aligned} \tag{25}$$

B.2.3. LOAD BALANCING

Load balancing (LB) involves assigning workloads to a minimal number of workers while respecting task constraints. Let there be M tasks and N workers. For task i , denote S_i as the set of workers capable of performing this task. Let y_{ij} represent the amount of workload assigned from task i to worker j . Let the binary variable x_j indicate whether the worker is in use. The LB problem can then be formulated as:

$$\begin{aligned}
 & \min \sum_{j=1}^N x_j \\
 & \text{s.t. } x_{ij} \leq a_i y_j, & \text{for } i = 1, \dots, M; j \in S_i \\
 & \sum_{i=1}^M \sum_{j \in S_i} y_{ij} \leq b_j, & \text{for } j = 1, \dots, N \\
 & \sum_{j \in S_i \setminus \{j'\}} y_{ij} \geq a_i, & \text{for } i = 1, \dots, M; j' \in S_i \\
 & x_j \in \{0, 1\}, & \text{for } j = 1, \dots, N \\
 & 0 \leq y_{ij} \leq b_j, & \text{for } i = 1, \dots, M; j \in S_i
 \end{aligned} \tag{26}$$

If we compare this formulation with the standard formulation, we have $p = N, n = N + \sum_{i=1}^M |S_i|$, where $|S_i|$ represents the number of items in this set. To convert this to standard form, the workload limits $y_{ij} \leq b_j$ are incorporated directly into the constraints.

B.3. Graph Neural Network Implementation Details

In our model, the neural network takes a graph $\mathcal{G}$ and multiple samples on this same graph to predict the transition probability. In this section, we will go into details about how the problem instances are encoded, and how the neural network gets implemented.

B.3.1. BIPARTITE GRAPH REPRESENTATION FOR MILP

Following previous works (Li et al., 2026), we represent MILPs as a special type of bipartite graph. More specifically, there are three types of nodes: integer variables, continuous variables, and constraints. Edges are drawn only between variable nodes (i.e., integer variables/continuous variables) and constraint nodes. Although there are three types of nodes, there are

only edges between the variable nodes and the constraint nodes. So this representation is still categorized as a bipartite graph.

The feature vector for integer variables $\mathbf{X}_{\text{Ivar}}$ includes the objective coefficient, the lower bound and upper bound of this variable, and indicators of whether it has an infinite upper/lower bound. If the lower/upper bound is infinite, the lower/upper bound is set to 0 and left for the indicator to encode the boundary information. Since we also explicitly predict integer assignments, the predicted solution is also taken as a feature.

As for continuous variables, since their prediction is implicit, their feature vectors $\mathbf{X}_{\text{Cvar}}$ have everything in the feature vectors of integer variables besides the predicted solution. Hence, the feature dimension of integer variables will be 6, and the feature dimension of continuous variables will be 5.

Constraints' feature vectors $\mathbf{X}_{\text{cons}}$ should encode everything about constraints. However, since we have already formulated the problems in concern into a standard form as in Eq. 7, we only need to encode the constants on the right-hand side. This also means that we will do a problem reformulation before putting it into the neural network. The dimension of $\mathbf{X}_{\text{cons}}$ is 1.

The edge encodes the algebraic relationship between variables and constraints. More specifically, the edge feature E encodes the coefficient of that variable in that constraint. The feature dimension is hence 1 for edges.

B.3.2. GRAPH NEURAL NETWORK

The input to our neural network is the aforementioned bipartite graph. To enhance the model's efficiency, we intend to process multiple samples at a time. On each graph, we sample K trajectories of length T on each graph, which makes KT training samples in total. In each iteration, the model processes B samples of the same graph at a time. In general, we follow the same model design as in (Li et al., 2026). However, the model in (Li et al., 2026) processes multiple graphs with one sample for each graph at a time. In our case, we intend to process multiple samples of the same graph in a batch. In the following paragraphs, we will detail the design of this modified GNN.

Model Input. There are 6 major parts of input:

- Integer variable features $\mathbf{X}'_{\text{Ivar}} \in \mathbb{R}^{n_{\text{Ivar}} \times 5}$. This only includes features related to the instance itself.
- Integer variable predictions $\mathbf{X}_{\text{IntSol}} \in \mathbb{R}^{n_{\text{Ivar}} \times B}$.
- Continuous variable features $\mathbf{X}_{\text{Cvar}} \in \mathbb{R}^{n_{\text{Ivar}} \times 5}$.
- Constraint features $\mathbf{X}_{\text{cons}} \in \mathbb{R}^{n_{\text{cons}} \times 1}$.
- Edge features (constraint coefficients) from integer variables to constraints $E_{\text{Ivar2cons}} \in \mathbb{R}^{n_{\text{Ivar}} \times n_{\text{cons}}}$.
- Edge features (constraint coefficients) from continuous variables to constraints $E_{\text{Cvar2cons}} \in \mathbb{R}^{n_{\text{Cvar}} \times n_{\text{cons}}}$.

where edge features are commonly represented in a sparse way.

Initial Embeddings. For continuous variables $\mathbf{X}_{\text{Cvar}}$ and constraints $\mathbf{X}_{\text{cons}}$, their initial embeddings are done directly through MLPs, i.e.,

$$\mathbf{h}'_{\text{Cvar}} = \text{MLP}_{\text{Cvar}}(\mathbf{X}_{\text{Cvar}}), \quad \mathbf{h}'_{\text{cons}} = \text{MLP}_{\text{cons}}(\mathbf{X}_{\text{cons}}), \quad \text{where } \mathbf{h}'_{\text{Cvar}} \in \mathbb{R}^{n_{\text{Cvar}} \times h}, \quad \mathbf{h}'_{\text{cons}} \in \mathbb{R}^{n_{\text{cons}} \times h} \quad (27)$$

with h denotes the hidden dimension. However, the encoding of integer variable features will be more complicated. The sampled predictions for integer variables are key features for integer variables. To increase $\mathbf{X}_{\text{IntSol}}$ to $\mathbf{X}'_{\text{Ivar}}$, $\mathbf{X}'_{\text{Ivar}}$ is first repeated B times. After concatenation, we have the final integer variable features $\mathbf{X}_{\text{Ivar}} \in \mathbb{R}^{n_{\text{Ivar}} \times B \times 6}$. Finally, a MLP is applied for embedding, i.e. $\mathbf{h}_{\text{Ivar}}^{(0)} = \text{MLP}_{\text{Ivar}}(\mathbf{X}_{\text{Ivar}})$, where $\mathbf{h}_{\text{Ivar}}^{(0)} \in \mathbb{R}^{n_{\text{Ivar}} \times B \times h}$.

For later processes, information from different samples should yield different features. Though initially, the embeddings of continuous variables and constraints are the same among different samples, after the solving begins, different features need to be extracted. Hence, they are also repeated to obtain the initial embedding $\mathbf{h}_{\text{Cvar}}^{(0)} \in \mathbb{R}^{n_{\text{Cvar}} \times B \times h}$ and $\mathbf{h}_{\text{cons}}^{(0)} \in \mathbb{R}^{n_{\text{cons}} \times B \times h}$.

Message Passing Layers. Since edge features are shared among all samples and we don't perform embedding on edge features, there is no need to replicate edge features as we do for continuous variables and constraint features. Instead, we

Table 4. Experiment results of different initialization techniques with post-processing methods

Datasets	Warmstart method	Neural Diving		Predict and Search		Apollo	
		Time ↓	Gap ↓	Time ↓	Gap ↓	Time ↓	Gap ↓
IP	Random	400s	11.55%	600s	14.50%	621s	11.94%
	Gurobi	400s + 5s	8.78%	600s + 5s	15.39%	800s + 5s	12.84%
	Ours	400s + 1.84s	-22.05%	600s + 1.84s	-22.33%	800s + 1.84s	-23.18%
LB	Random	333s	3.00%	29.61s	5.82%	20.84s	10.06%
	Gurobi	400s + 20s	0.21%	600s + 20s	0.07%	800s + 20s	0.45%
	Ours	400s + 24.62s	0.01%	372s + 24.62s	-0.01%	800s + 24.62s	-0.01%

rely on feature broadcasting to incorporate edge information directly into the message passing over variable nodes. Under this design, we can directly adopt the tripartite and bipartite GCN convolution layers proposed in (Li et al., 2026). The message-passing procedure at each layer is formulated as follows:

$$\begin{aligned} \mathbf{h}_{\text{con}}^{(l)} &= \mathbf{h}_{\text{con}}^{(l-1)} + \text{MLP}_{\text{con}}^{(l)}(\text{TriConv}(\mathbf{h}_{\text{Ivar}}^{(l-1)}, \mathbf{h}_{\text{Cvar}}^{(l-1)}, \mathbf{h}_{\text{cons}}^{(l-1)}, E_{\text{Ivar2cons}}, E_{\text{Cvar2cons}})) \\ \mathbf{h}_*^{(l)} &= \mathbf{h}_*^{(l-1)} + \text{MLP}_*^{(l)}(\text{BiConv}(\mathbf{h}_{\text{cons}}^{(l-1)}, \mathbf{h}_*^{(l-1)}, E_{\text{Ivar2cons}}, E_{\text{cons2*}})), \text{ where } * \in \{\text{Ivar}, \text{Cvar}\} \end{aligned} \quad (28)$$

where $E_{\text{cons2*}}$ represents the reverse edges, which can be easily inferred from $E_{*2\text{cons}}$.

Output Layer. Our model is focused on predicting integer variables, leaving continuous variables to the remaining linear programming problems. Hence, we only need an MLP to output the final prediction for the integer variables, i.e.,

$$\mathbf{o}_{\text{Ivar}} = \text{MLP}_{\text{out}}^{\text{Ivar}}(\mathbf{h}_{\text{Ivar}}^{(L)}), \text{ where } \mathbf{o}_{\text{Ivar}} \in \mathbb{R}^{n_{\text{Ivar}} \times 1} \quad (29)$$

Since all the MILPs in concern are binary, we can directly output the prediction logits for those integer variables.

C. Additional Experiment Results

C.1. Ablation Studies

In this section, we investigate the extent to which the observed performance gains can be attributed to the post-processing procedures. To disentangle the effect of our sampler as a warm-start generator from that of the downstream solvers, we allocate Gurobi a comparable short time budget—matched to the neural inference cost—to produce alternative warm-start solutions, which are then provided to the same downstream solvers. We additionally include randomly initialized solutions as a baseline.

The results are summarized in Table 4. Performance is evaluated in terms of both the total runtime (i.e., warm-start generation time plus downstream solving time) and the optimality gap with respect to the reference solutions. Across both datasets, warm-starts generated by our method consistently yield superior downstream performance compared to those initialized by short-budget Gurobi runs or random initialization.

These findings indicate that the improvement cannot be solely attributed to the capability of the downstream solvers. Rather, the quality and structural properties of the initial solutions generated by our sampler play a crucial role. In particular, our sampler tends to produce solutions that are more favorable for subsequent local refinement, enabling downstream search procedures to more effectively exploit the solution landscape.

C.2. Train/Inference Costs

The training time per epoch is reported at Table 5, and each model is trained for 50 epochs.

	CFLP-(100,100)	CFLP-(200,200)	IP	LB
Train time per epoch	40m	1.5h	20m	1h
Test time per instance	5s	30s	2s	25s

Table 5. Training and testing time comparison.